Tutorial: Make This Project a Live GitHub Web Page (12th Class Friendly)

1. What you are building

You are creating a live project website on GitHub Pages.

Think of it like this:

- Python model is the engine.
- GitHub Pages site is the showroom.

The showroom explains and demonstrates your project to teachers, recruiters, and clients.

2. Very important limitation

GitHub Pages can host only static files:

- HTML
- CSS
- JavaScript
- images

GitHub Pages cannot run:

- Python
- Streamlit server
- FastAPI server

So your live page will show:

- clear explanation of the ML pipeline
- results and metrics
- screenshots
- instructions to run dashboard/API locally

This is the correct professional approach for GitHub Pages portfolios.

3. What is already prepared for you in this folder

Inside github_pages_demo:

- complete static site in site
- GitHub Actions deployment template in workflow-template/deploy-pages.yml
- setup script setup_for_repo.ps1
- PDF generator generate_github_pages_tutorial_pdf.py

4. One-time setup before deployment

4.1 Create GitHub repository

1. Go to github.com
2. Create a new repository
3. Example name: image-classifier-portfolio
4. Make it Public (for easy live demo sharing)

4.2 Push your local project

From project root:

```powershell
git init
git add .
git commit -m "Initial portfolio project"
git branch -M main
git remote add origin https://github.com/<username>/<repo-name>.git
git push -u origin main
```

```
If repository already exists, just do add/commit/push.
```

```
## 5. Install GitHub Pages workflow automatically
```

```
From project root, run:
```

```
```powershell
powershell -ExecutionPolicy Bypass -File github_pages_demo/setup_for_repo.ps1
```

What this script does:

- creates .github/workflows if missing
- copies workflow-template/deploy-pages.yml

to .github/workflows/deploy-pages.yml

Then commit and push:

```powershell
git add .github/workflows/deploy-pages.yml
git commit -m "Add GitHub Pages deployment workflow"
git push
```

```
## 6. Enable Pages in GitHub
```
```

1. Open your repository on GitHub.
2. Go to Settings.
3. Open Pages.
4. Under Build and deployment:
- Source: GitHub Actions
5. Save.

Now each push to main will deploy your site.

7. How deployment works (simple)

When you push code:

1. GitHub Actions runs workflow
[deploy-pages.yml](workflow-template/deploy-pages.yml).
2. It takes files from [github_pages_demo/site](site).
3. It publishes them to GitHub Pages hosting.
4. You get a public URL.

URL format:

- https://<username>.github.io/<repo-name>/

8. Understanding the prepared website files

8.1 site/index.html

Contains:

- hero section (project title)
- architecture flow
- metrics cards
- links to dashboard/API commands
- repository links

8.2 site/styles.css

Controls:

- colors
- spacing
- cards
- mobile responsive layout

8.3 site/script.js

Sets small dynamic content like:

- deployment year
- sample metrics values

8.4 site/.nojekyll

Tells GitHub Pages not to process site through Jekyll.

Useful for clean static hosting and predictable paths.

9. How to customize for your own branding

Edit site/index.html:

- your name
- project summary
- links

Edit site/script.js:

- validation accuracy
- F1 score
- model size
- latency

Edit site/styles.css:

- colors
- typography
- card style

10. Add proof for stronger demo

Good additions:

- screenshot of Streamlit dashboard
- screenshot of API docs (/docs)
- sample JSON response from /predict
- model metrics table from reports/training_log.csv

Place images inside site/assets and reference them in index.html.

11. Optional: make dashboard live too

Since GitHub Pages cannot run Streamlit, use one of these:

- Streamlit Community Cloud
- Hugging Face Spaces
- Render / Railway / Azure / AWS

Then place that URL as "Live Interactive Demo" button in index.html.

Best practice:

- GitHub Pages = portfolio homepage
- Streamlit/FastAPI host = interactive backend demo

12. Troubleshooting

Problem: workflow not running

- check file path is exactly .github/workflows/deploy-pages.yml
- check branch is main

Problem: 404 page

```
- check repository is public
- check Pages source is GitHub Actions
- wait 1-3 minutes after successful deploy

Problem: CSS/JS not loading
- use relative paths like ./styles.css and ./script.js
- avoid absolute localhost links in HTML

Problem: old content still visible
- hard refresh browser (Ctrl+F5)

## 13. Viva/interview explanation (short)

Say this:

"I separated model execution from portfolio hosting.
GitHub Pages hosts a static but professional demo website.
The trained model remains in Python and is exposed via Streamlit and FastAPI.
This architecture is realistic and production-friendly because frontend hosting and backend inference are decoupled."

## 14. Final checklist

- repository pushed to GitHub
- workflow copied to .github/workflows/deploy-pages.yml
- Pages source set to GitHub Actions
- action run is green
- live URL opens
- README contains live URL

You now have a public live demo page for this ML project.
```